



NITTE
(Deemed to be University)

| **NMAM INSTITUTE
OF TECHNOLOGY**

Nitte (DU) established under Section 3 of UGC Act 1956 | Accredited with 'A+' Grade by NAAC

Introduction To Python Programming

25CSE102



Programming is a skill best
acquired by practice and
example rather than from books.

Alan Turing

Course Outcomes:

At the end of the course student will be able to

1. Understand basic Python programming constructs.
2. Develop modular programs using Python functions.
3. Utilize Python data structures.
4. Perform file operations and apply object-oriented programming.
5. Handle exceptions and visualize data.

Textbooks:

1. Kenneth A. Lambert, “The Fundamentals of Python: First Programs”, Cengage Learning, 2nd Edition, 2023.
2. Yashwanth Kanetkar, Adithya Kanetkar, “Let us Python”, 6th Edition, 2023.
3. Al Sweigart, —Automate the Boring Stuff with Python^{ll}, 1st Edition, No Starch Press, 2015.
4. Mark Summerfield, “Programming in Python 3 - A Complete Introduction to the Python Language”, Second Edition, Addison-Wesley, 2009.
5. Y. Daniel Liang, “Introduction to Programming Using Python”, Pearson, 2017.

Unit -1

Introduction to Python and Function

Chapter 1:Python Concepts



**Chapter 2: Controls and
Looping statements**



What is python?

Python is a very popular general-purpose interpreted, interactive, object-oriented, and high-level programming language.

Why to learn python?

- Python is Open Source which means its available free of cost.
- Python is simple and so easy to learn Python is versatile and can be used to create many different things.
- Python has powerful development libraries include AI, ML etc.

Python is a widely used high-level, interpreted programming language.

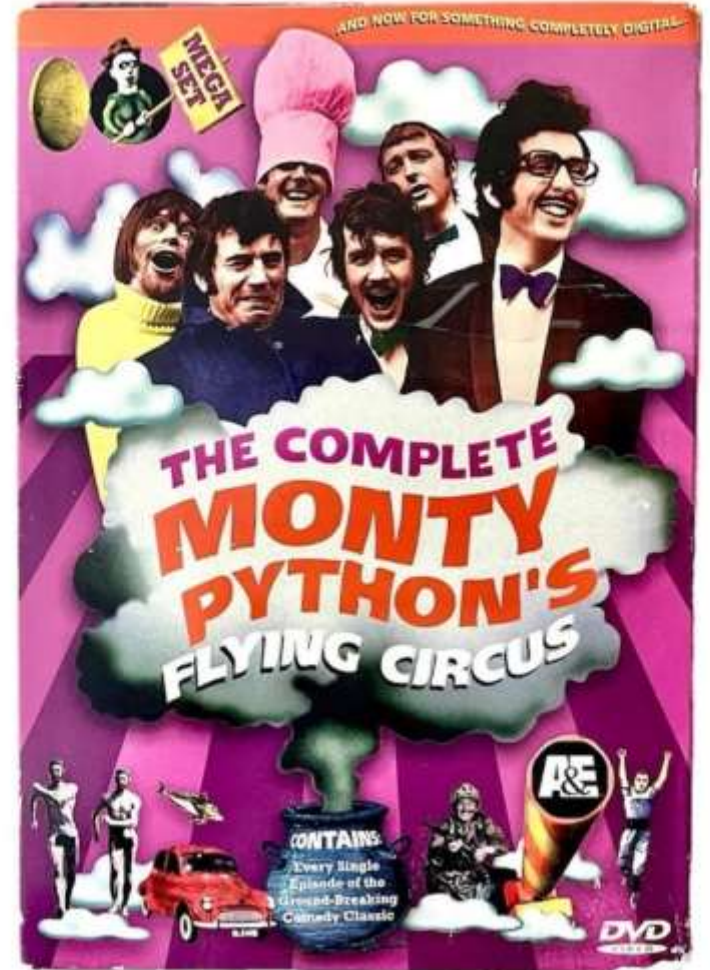
It was created by **Guido van Rossum** in 1991 and further developed by the Python Software Foundation



Why the name python?

Guido van Rossum was interested in watching a comedy show, which is telecasting on the BBC channel from 1969 to 1974 “**The complete Monty Python’s Flying Circus**”.

Guido Van Rossum thought he needed a name that was short, unique, and slightly mysterious for his programming language, so he decided to call the language **Python**.



Applications of Python Programming

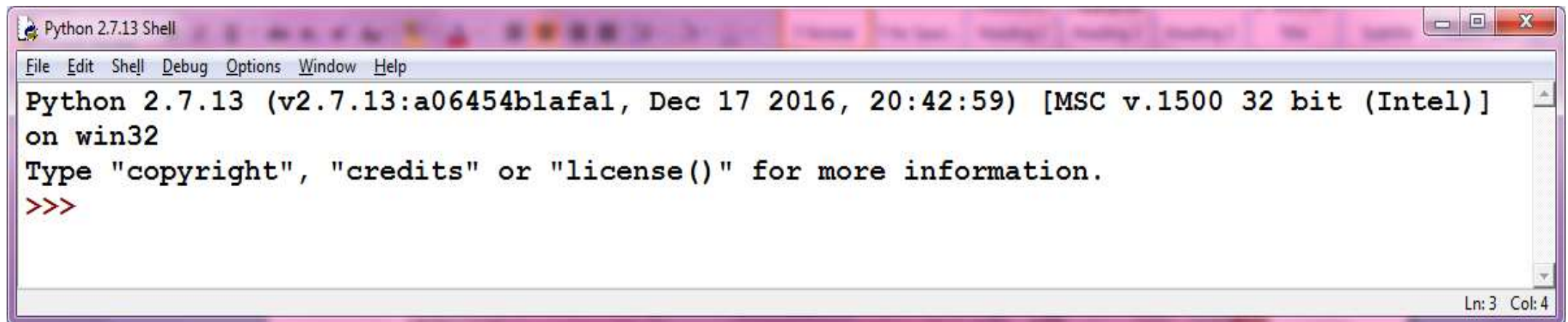
- Web Development
- Game Development
- Scientific and Numeric applications
- Artificial Intelligence and Machine Learning based applications
- Data Science related applications
- Desktop GUI applications
- Software Development
- Enterprise-level/Business Applications
- Education programs and training courses
- Web Scraping Applications
- Image Processing and Graphic Design Applications
- Data Analysis

Key Features of Python

- **Python is Easy to Learn and Use:** There is no prerequisite to start Python, since it is Ideal programming language for beginners.
- **High Level Language:** Python don't let you worry about low-level details, like memory management, hardware-level operations etc.
- **Python is Interpreted:** Code is executed line-by-line directly by interpreter, and no need for separate compilation. Which means –
 - You can run the same code across different platforms.
 - You can make the changes in code without restarting the program.

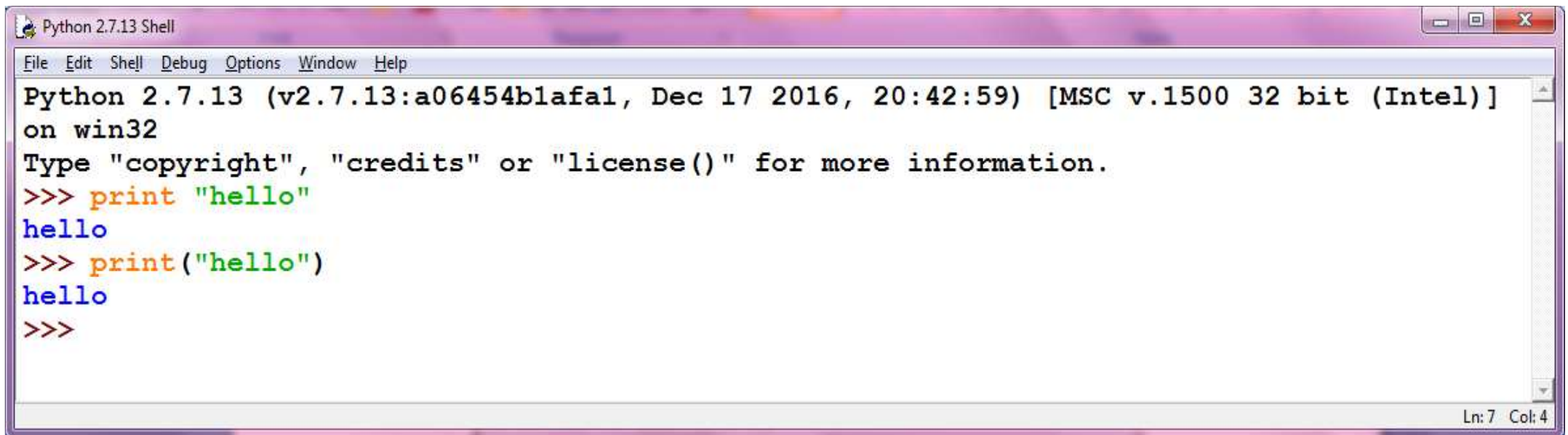
a) Demonstrate about Basics of Python Programming.

- Python comes with an interactive interpreter.
- When you type **python** in your shell or command prompt, the python interpreter becomes active with a
 >>> and waits for your commands.



```
Python 2.7.13 Shell
File Edit Shell Debug Options Window Help
Python 2.7.13 (v2.7.13:a06454b1afa1, Dec 17 2016, 20:42:59) [MSC v.1500 32 bit (Intel)]
on win32
Type "copyright", "credits" or "license()" for more information.
>>>
```

The screenshot shows a Windows-style application window titled "Python 2.7.13 Shell". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area displays the Python 2.7.13 startup message, including the version, build date, and architecture. It ends with the interactive prompt ">>>". The status bar at the bottom right indicates "Ln: 3 Col: 4".

A screenshot of a Python 2.7.13 Shell window. The window has a title bar with the text "Python 2.7.13 Shell" and standard Windows window controls (minimize, maximize, close). Below the title bar is a menu bar with the following items: File, Edit, Shell, Debug, Options, Window, and Help. The main area of the window contains the following text:

```
Python 2.7.13 (v2.7.13:a06454b1afa1, Dec 17 2016, 20:42:59) [MSC v.1500 32 bit (Intel)]  
on win32  
Type "copyright", "credits" or "license()" for more information.  
>>> print "hello"  
hello  
>>> print("hello")  
hello  
>>>
```

The text is color-coded: "print" is orange, "hello" is green, and the prompt ">>>" is blue. The output "hello" is blue. At the bottom right of the window, there is a status bar that reads "Ln: 7 Col: 4".

ii. Running Python Scripts in IDLE (Integrated Development and Learning Environment):

- IDLE is the standard Python development environment.
- It's name is an acronym of "**Integrated DeveLopment Environment**".
- It works well on both Unix and Windows platforms.
- It has a Python shell window, which gives you access to the Python interactive mode.
- It also has a file editor that lets you create and edit existing Python source files.

- Goto File menu click on New File (CTRL+N) and write the code and save add.py

iii. Running Python scripts in Command Prompt:

- Before going to run **python27** folder in the command prompt.
- Open your text editor, type the following text and save it as **hello.py**.
`print "hello"`
- In the command prompt, and run this program by calling `python hello.py`.
- Make sure you change to the directory where you saved the file before doing it.

C:\Windows\system32\cmd.exe

```
C:\Python27>python Hello.py  
Hello, Mothilal
```

```
C:\Python27>
```

Python Identifier

- Python Identifier is the name we give to identify a variable, function, class, module or other object.
- That means whenever we want to give an entity a name, that's called identifier.

Python Keywords

- Python keywords are the words that are reserved.
- That means you can't use them as name of any entities like variables, classes and functions.
- **There were 33 keywords in Python 3.7. However, the number increased to 35 in Python 3.8.**

False
None
True
and
as
assert
break
class
continue

def
del
elif
else
except
finally
for
from
global

if
import
in
is
lambda
nonlocal
not
or
pass

raise
return
try
while
with
yield

Keyword	Description	Keyword	Description
and	A logical operator	from	To import specific parts of a module
as	To create an alias	global	To declare a global variable
assert	For debugging	if	To make a conditional statement
break	To break out of a loop	import	To import a module
class	To define a class	in	To check if a value is present in a list, tuple, etc.
continue	To continue to the next iteration of a loop	is	To test if two variables are equal
def	To define a function	lambda	To create an anonymous function
del	To delete an object	None	Represents a null value
elif	Used in conditional statements, same as else if	nonlocal	To declare a non-local variable
else	Used in conditional statements	not	A logical operator
except	Used with exceptions, what to do when an exception occurs	or	A logical operator
False	Boolean value, result of comparison operations	pass	A null statement, a statement that will do nothing
finally	Used with exceptions, a block of code that will be executed no matter if there is an exception or not	for	To create a for loop

Variable

- A variable, as the name indicates is something whose value is changeable over time.
- In fact a variable is a memory location where a value can be stored. Later we can retrieve the value to use.
- Python has no command for declaring a variable.
- A variable is created the moment you first assign a value to it.
- Example

```
x = 5
y = "John"
print(x)
print(y)
```
- No need to define type of variable.
- During the execution, the type of a variable is inferred from the context in which it is being used. Hence Python is called **dynamically-typed language**.

- Simple variable assignment

```
a = 10
```

```
pi = 3.14
```

```
name = 'Sanjay'
```

```
x = 4          # x is of type int
x = "nitte"    # x is now of type str
print(x)
```

```
x = str(3)     # x will be '3'
y = int(3)     # y will be 3
z = float(3)   # z will be 3.0
```

```
x = 5  
y = "John"  
print(type(x))  
print(type(y))
```

Multiple variable assignment

```
a = 10 ; pi = 31.4 ; name = 'Sanjay'  # use ; as statement separator  
a, pi, name = 10, 3.14, 'Sanjay'  
a = b = c = d = 5
```

Rules for Writing Variable / Identifiers

There are some rules for writing Identifiers. But first you must know Python is case sensitive. That means **Name** and **name** are two different identifiers in Python. Here are some rules for writing Identifiers in python.

1. Identifiers can be combination of uppercase and lowercase letters, digits or an underscore (_).
So **myVariable**, **variable_1**, **variable_for_print** all are valid python identifiers.
2. An Identifier can not start with digit. So, while **variable1** is valid, **1variable** is not valid.
3. We can't use special symbols like **!,#,@,%,\$** etc in our Identifier.
4. Identifier can be of any length.

Whether the following statement is true?

```
x, y, z = "apple", "banana", "Cherry"
```

```
print(x)
```

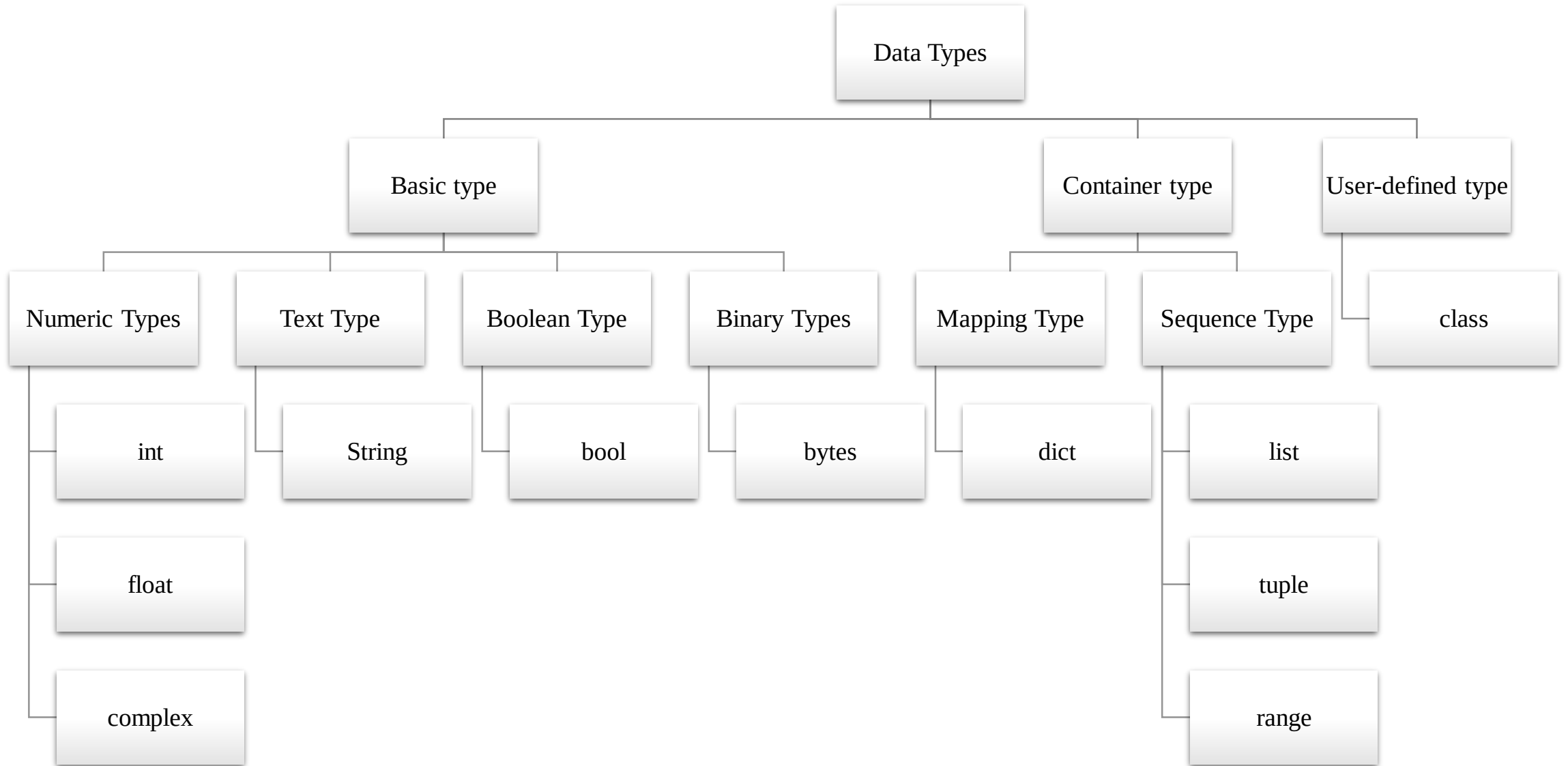
```
print(y)
```

```
print(z)
```

Python Data Types

- Variables can store data of different types, and different types can do different things.
- Python has the following data types built-in by default, in these categories:
- 3 categories of data types:
 1. Basic types: int, float, complex, bool, string, bytes
 2. Container types: list, tuple, set, dict
 3. User-defined types: Class

Categories	
Text Type:	str
Numeric Types:	int, float, complex
Sequence Types:	list, tuple, range
Mapping Type:	dict
Boolean Type:	bool
Binary Types:	bytes, bytearray
None Type:	NoneType



- `int` can be expressed in binary, decimal, octal, hexadecimal.

Integer type	Starts with	Example
decimal	0-9	156
binary	0b/0B	0b101
octal	0o/0O	0o432
hexadecimal	0x/0X	0x443

- `int` can be of any arbitrary size
- Python has arbitrary precision integers. Hence, we can create as big integers as we want.
- Arithmetic operations can be performed on integers without worrying about overflow/underflow.

```
a = 123
```

```
b = 1234567890
```

```
c = 123456789012345678901234567890
```

- **float** can be expressed in fractional or exponential form
 - Eg: -314.1528, 3.141528e2, 3.141528E2
- **complex** contains real and imaginary part
 - 3+2j, 1+4J
- **bool** can take any of 2 Boolean values both starting in caps
 - True, False
- **string** is an immutable collection of Unicode characters enclosed within ' ', " " or """ """
 - 'hi', "hi", """hi"""
- **bytes** represent binary data
 - B'\xa1\xe4\x56' -represents 3bytes with hex value a1e456

Input and Output

- **print()**- used to output values on the screen
- **input()** built-in function can be used to retrieve input values from keyboard.
- input() function returns a string
 - i.e if 23 is entered it returns '23'
- So to perform arithmetic operations on the number entered . Convert string to int or float.

```
n = input('Enter your name: ')
age = int(input('Enter your age: '))
salary = float(input('Enter your salary: '))
print(name, age, salary)
```

Operators

Python divides the operators in the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators

1.Arithmetic operators

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponent	$x ** y$
//	Floor division	$x // y$

Operation Nuances

- On performing floor division, $a//b$, result is largest integer which is less than or equal to the quotient.

```
print(10 // 3)    # yields 3
print(-10 // 3)   # yields -4
print(10 // -3)   # yields -4
print(-10 // -3)  # yields 3
print(3 // 10)    # yields 0
print(3 // -10)   # yields -1
print(-3 // 10)   # yields -1
print(-3 // -10)  # yields 0
```

In $-10 // 3$, multiple of 3 which will yield -10 is -3.333, whose floor value is -4.

In $10 // -3$, multiple of -3 which will yield 10 is -3.333, whose floor value is -4.

In $-10 // -3$, multiple of -3 which will yield -10 is 3.333, whose floor value is 3.

2. Comparison Operators

- These are used to compare two operands. For example, compare the ages of two persons, compare the prices of two or more items, etc.
- They result in either a TRUE (Non-zero) value or FALSE (Zero) value

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

3.Logical operators

- The result of these operators is either TRUE (ONE) or FALSE (ZERO)

Operator	Description	Example
and	Returns True if both statements are true	$x < 5$ and $x < 10$
or	Returns True if one of the statements is true	$x < 5$ or $x < 4$
not	Reverse the result, returns False if the result is true	not($x < 5$ and $x < 10$)

4. Identity operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

```
x = ["apple", "banana"]
```

```
y = ["apple", "banana"]
```

```
z = x
```

- `print(x is z)` `# returns True because z is the same object as x`
- `print(x is y)` `# returns False because x is not the same object as y, even if they have the same content`
- `print(x == y)` `# to demonstrate the difference between "is" and "==": this comparison returns True because x is equal to y`

5.Membership operators

- Membership operators are used to test if a sequence is presented in an object

Operator	Description	Example	
in	Returns True if a sequence with the specified value is present in the object	x in y	x = ["apple", "grapes"] print("apple" in x) Output: TRUE
not in	Returns True if a sequence with the specified value is not present in the object	x not in y	x = ["apple", "grapes"] print("pineapple" not in x) Output: TRUE

6. Bitwise operators

- Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

- Take two bit values X and Y, where $X = 7 = (111)_2$ and $Y = 4 = (100)_2$.
Take Bitwise and of both X & y

- Note:** Here, $(111)_2$ represent binary number.

$$\begin{array}{r} 111_2 \\ \& 100_2 \\ \hline 100_2 = 4 \end{array}$$

- Take two bit values X and Y, where $X = 7 = (111)_2$ and $Y = 4 = (100)_2$.
Take Bitwise OR of both X, Y

$$\begin{array}{r} 111_2 \\ | 100_2 \\ \hline 111_2 = 7 \end{array}$$

Take two bit values X and Y, where $X = 7 = (111)_2$ and $Y = 4 = (100)_2$.
Take Bitwise and of both X & Y

$$\begin{array}{r} 111_2 \\ \wedge 100_2 \\ \hline 011_2 = 3 \end{array}$$

Take two bit values X and Y, where $X = 5 = (101)_2$. Take Bitwise NOT of X.

$$\begin{array}{r} \sim 1001_2 \\ \hline 0110_2 = 6 \end{array}$$

- **Example 1:**

$a = 5 = 0000\ 0101$ (Binary)

$a \ll 1 = 0000\ 1010 = 10$

$a \ll 2 = 0001\ 0100 = 20$

Example 2:

$b = -10 = 1111\ 0110$ (Binary)

$b \ll 1 = 1110\ 1100 = -20$

$b \ll 2 = 1101\ 1000 = -40$

Example 1:

$a = 10 = 0000\ 1010$ (Binary)

$a \gg 1 = 0000\ 0101 = 5$

Example 2:

$a = -10 = 1111\ 0110$ (Binary)

$a \gg 1 = 1111\ 1011 = -5$

7.Assignment operators

- Assignment operators are used to assign values to variables

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3

Operator	Example	Same As
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Precedence and Associativity

- Precedence(priority)-multiple operators in an expression
- Associativity- operators with same precedence
- Left to right associativity or right to left associativity
- **$c = a * b / c$**
 - Same precedence
 - $*$ is done before $/$ (arithmetic operators have Left to right associativity)

Description	Operator	Associativity
Grouping	()	Left to Right
Function call	function()	Left to Right
Slicing	[start:end:step]	Left to Right
Exponentiation	**	Right to Left
Bitwise NOT	~	Right to Left
Unary plus / minus	+ -	Left to Right
Multiplication	*	Left to Right
Division	/	Left to Right
Modular Division	%	Left to Right
Addition	+	Left to Right
Subtraction	-	Left to Right
Bitwise left shift	<<	Left to Right
Bitwise right shift	>>	Left to Right
Bitwise AND	&	Left to Right
Bitwise XOR	^	Left to Right
Bitwise OR		Left to Right
Membership	in not in	Left to Right
Identity	is is not	Left to Right
Relational	< > <= >=	Left to Right
Equality	==	Left to Right
Inequality	!= <>	Left to Right
Logical NOT	not	Left to Right
Logical AND	and	Left to Right
Logical OR	or	Left to Right
Assignment	= += -= *= /= %= //= **= &= = ^= >>= <<=	Right to Left

Type Conversions

- Mixed mode operations:
 - Operation between int and float will yield float.
 - Operation between int and complex will yield complex.
 - Operation between float and complex will yield complex.
- We can convert one numeric type to another using built-in functions.
 - **int() , float(), complex(), bool()**

- `int()` - constructs an integer number from an integer literal, a float literal (by removing all decimals), or a string literal (providing the string represents a whole number)

- `x = int(1)` # x will be 1
 # y will be 2
`y = int(2.8)` # z will be 3
`z = int("3")`

- `float()` - constructs a float number from an integer literal, a float literal or a string literal (providing the string represents a float or an integer)

- `x = float(1)` # y will be 2.8
 # z will be 3.0
`y = float(2.8)` # w will be 4.2
`z = float("3")`
`w = float("4.2")`

- `str()` - constructs a string from a wide variety of data types, including strings, integer literals and float literals

- `x = str("s1")` # x will be 's1'

`y = str(2)` # y will be '2'

`z = str(3.0)` # z will be '3.0'

Conversion	Explanation
<code>int(float/numeric string)</code>	from float/numeric string to int
<code>int(numeric string, base)</code>	from numeric string to int in base
<code>float(int/numeric string)</code>	from int/numeric string to float
<code>float (int)</code>	from int to float
<code>complex(int/float)</code>	Convert to complex with imaginary part zero
<code>complex(int/float, int/float)</code>	Convert to complex
<code>bool(int/float)</code>	from int to float to True/False
<code>str(int/float/bool)</code>	Convert to string
<code>chr(int)</code>	Yields a character corresponding to int

Built-in functions

- `print()` is also a built-in function.
- Help about any built-in function is available using **`help(function)`**
- **Built-in function commonly used with numbers are:**

<code>abs(x)</code>	# returns absolute value of x	<code>max(x1, x2,...)</code>	# returns largest argument
<code>pow(x, y)</code>	# returns value of x raised to y	<code>divmod(x, y)</code>	# returns a pair(<code>x // y</code> , <code>x % y</code>)
<code>min(x1, x2,...)</code>	# returns smallest argument	<code>round(x [,n])</code>	# returns x rounded to n digits after .
		<code>bin(x)</code>	# returns binary equivalent of x
		<code>oct(x)</code>	# returns octal equivalent of x
		<code>hex(x)</code>	# returns hexadecimal equivalent of x

```
a = abs(-3)           # assigns 3 to a
print(min(10, 20, 30, 40)) # prints 10
print(hex(26))         # prints 1a
```

Built-in Modules

- Each module contains many functions.
 - math, cmath, random, decimal

math - many useful mathematics functions.

cmath - functions for performing operations on complex numbers.

random - functions related to random number generation.

decimal - functions for performing precise arithmetic operations.

- Mathematical functions in **math** module:

<code>pi, e</code>	<code># values of constants pi and e</code>
<code>sqrt(x)</code>	<code># square root of x</code>
<code>factorial(x)</code>	<code># factorial of x</code>
<code>fabs(x)</code>	<code># absolute value of float x</code>
<code>log(x)</code>	<code># natural log of x (log to the base e)</code>
<code>log10(x)</code>	<code># base-10 logarithm of x</code>
<code>exp(x)</code>	<code># e raised to x</code>
<code>trunc(x)</code>	<code># truncate to integer</code>
<code>ceil(x)</code>	<code># smallest integer >= x</code>
<code>floor(x)</code>	<code># largest integer <= x</code>
<code>modf(x)</code>	<code># fractional and integer parts of x</code>

- **round()** built-in function can round to a specific number of decimal places, whereas
- **trunc(), ceil(), floor()** always round to zero decimal places.

- Trigonometric functions in **math** module:

<code>degrees(x)</code>	<code># radians to degrees</code>
<code>radians(x)</code>	<code># degrees to radians</code>
<code>sin(x)</code>	<code># sine of x radians</code>
<code>cos(x)</code>	<code># cosine of x radians</code>
<code>tan(x)</code>	<code># tan of x radians</code>
<code>sinh(x)</code>	<code># hyperbolic sine of x</code>
<code>cosh(x)</code>	<code># hyperbolic cosine of x</code>
<code>tanh(x)</code>	<code># hyperbolic tan of x</code>
<code>acos(x)</code>	<code># cos inverse of x, in radians</code>
<code>asin(x)</code>	<code># sine inverse of x, in radians</code>
<code>atan(x)</code>	<code># tan inverse of x, in radians</code>
<code>hypot(x, y)</code>	<code># sqrt(x * x + y * y)</code>

- Random number generation functions from **random** module:

```
random( )           # random number between 0 and 1  
randint(start, stop) # random number in the range  
seed( )             # sets current time as seed for random number generation  
seed(x)              # sets x as seed for random number generation logic
```

- To use functions present in a module, we need to import the module using **import** statement:

```
import math  
import random  
print(math.factorial(5))           # prints 120  
print(math.degrees(math.pi))      # prints 180.0  
print(random.random( ))           # prints 0.8960522546341796
```

- To get list of functions in each built-in module:

```
import math  
print(dir(__builtins__))    # 2 underscores before and after builtins
```

```
print(dir(math))
```

Comments in python

What are Comments?

- Comments are used to explain the code and make it easier to read.
- Comments are ignored during execution.
- **Single line comment begin with #**
#Program to add 2 numbers
a=b+c

- **Multiline comment written in a pair of `'''` or `"""`**

```
""" This function calculates the greatest common divisor (GCD)
of two numbers using the Euclidean algorithm. The GCD of two
numbers is the largest number that divides both of them without
leaving a remainder. """
```

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

result = gcd(48, 18)
```

Indentation

- ‘Unexpected indent’

```
a = 20
```

```
    b = 45
```

Control statements

- So far statements in all our programs got executed sequentially or one after the other.
- Sequence of execution of instructions in a program can be altered using:
 - Decision control instruction
 - Repetition control instruction

- Colon(:) after if, else, elif is compulsory.
- Statements in if block, else block, elif block have to be indented.
- Indented statements are treated as block of statements.
- Used to group statements. Use either 4 spaces or a tab for indentation.

If statement:

The if statement is used to test a specific condition. If the condition is true, a block of code (if-block) will be executed.

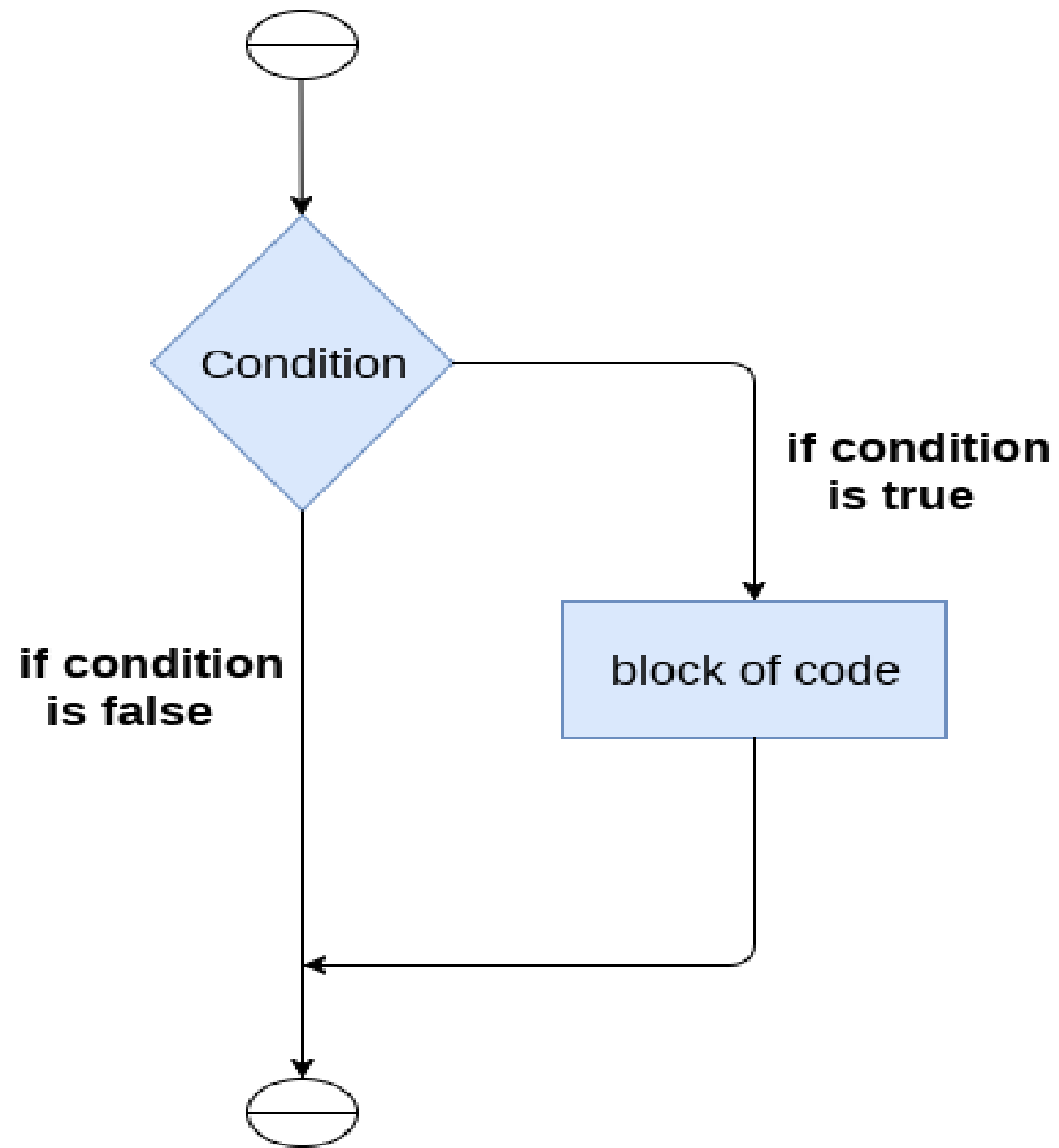
Syntax:

if condition:

 statement1

 statement2

statement3



Example:

```
a = 30
```

```
b = 100
```

```
if b > a:
```

```
    print("b is greater than a")
```

```
print("program ends here..")
```

The if-else statement

The if-else statement is similar to if statement except the fact that, it also provides the block of the code for the false case of the condition to be checked. If the condition provided in the if statement is false, then the else statement will be executed.

Syntax:

if condition:

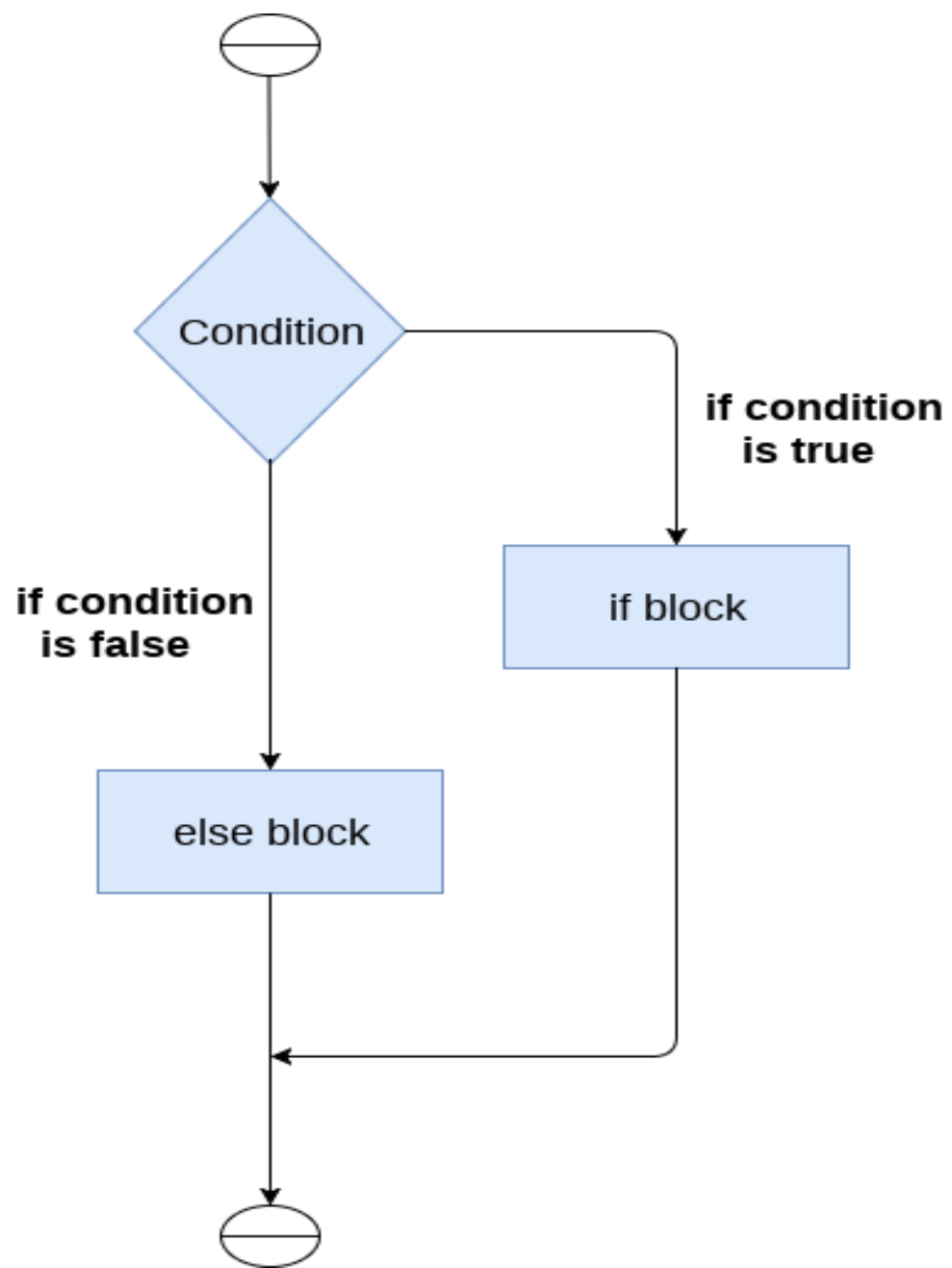
 statement1

 statement2

else:

 statement3

 statement4



Example:

a = 30

b = 100

if b > a:

 print("b is greater than a")

else:

 print("a is greater than b")

print("program ends here..")

The elif statement:

- The elif statement enables us to check multiple conditions and execute the specific block of statements depending upon the true condition among them.
- We can have any number of elif statements in our program depending upon our need.

Syntax:

if condition1:

 statement1

 statement2

elif condition2 :

 statement3

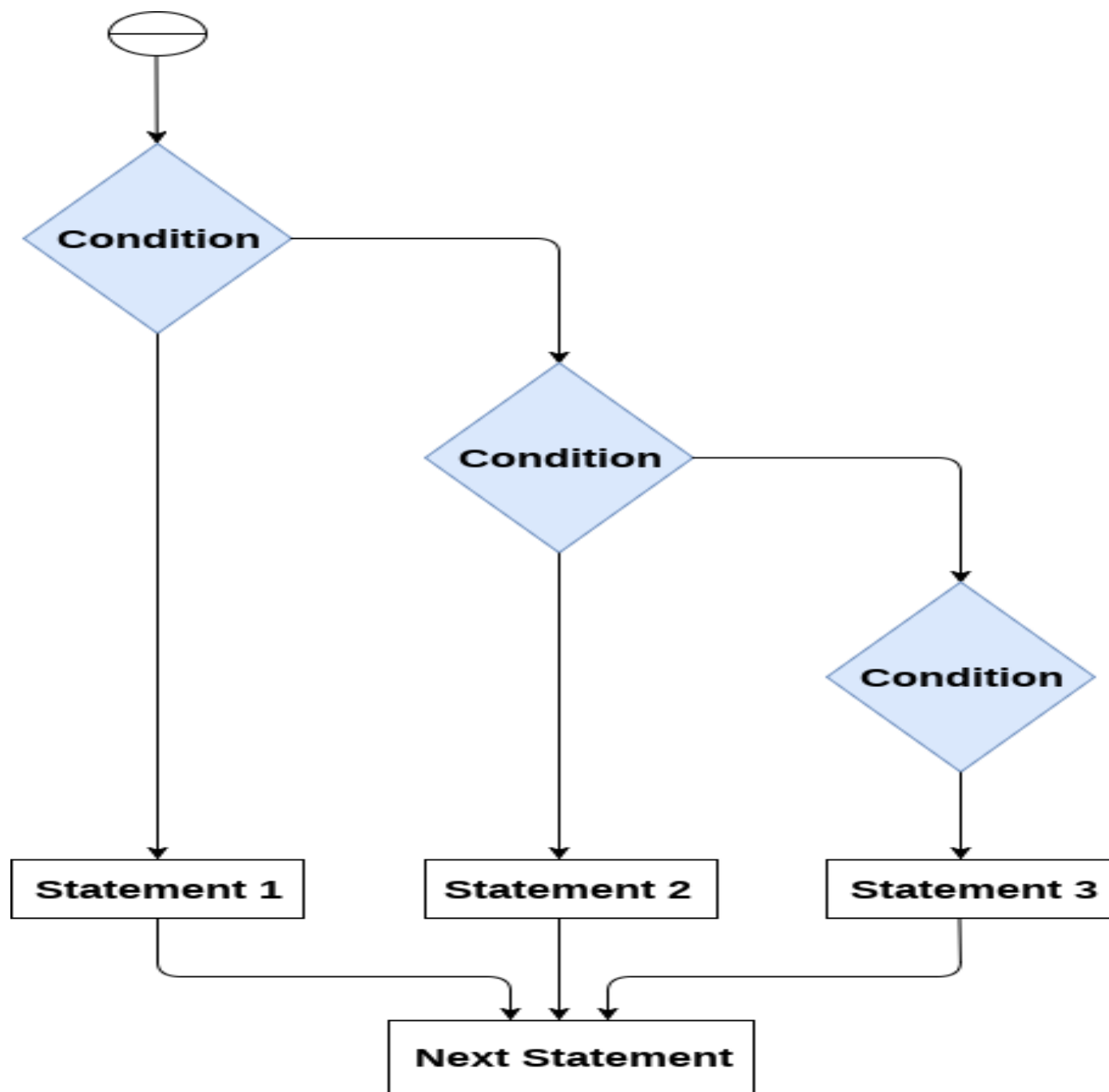
elif condition3 :

 statement4

else :

 statement4

next statement



Example:

a = 30

b = 100

if a > b:

 print("a is greater than b")

elif a < b:

 print("a is smaller than b")

else:

 print("a and b are equal")

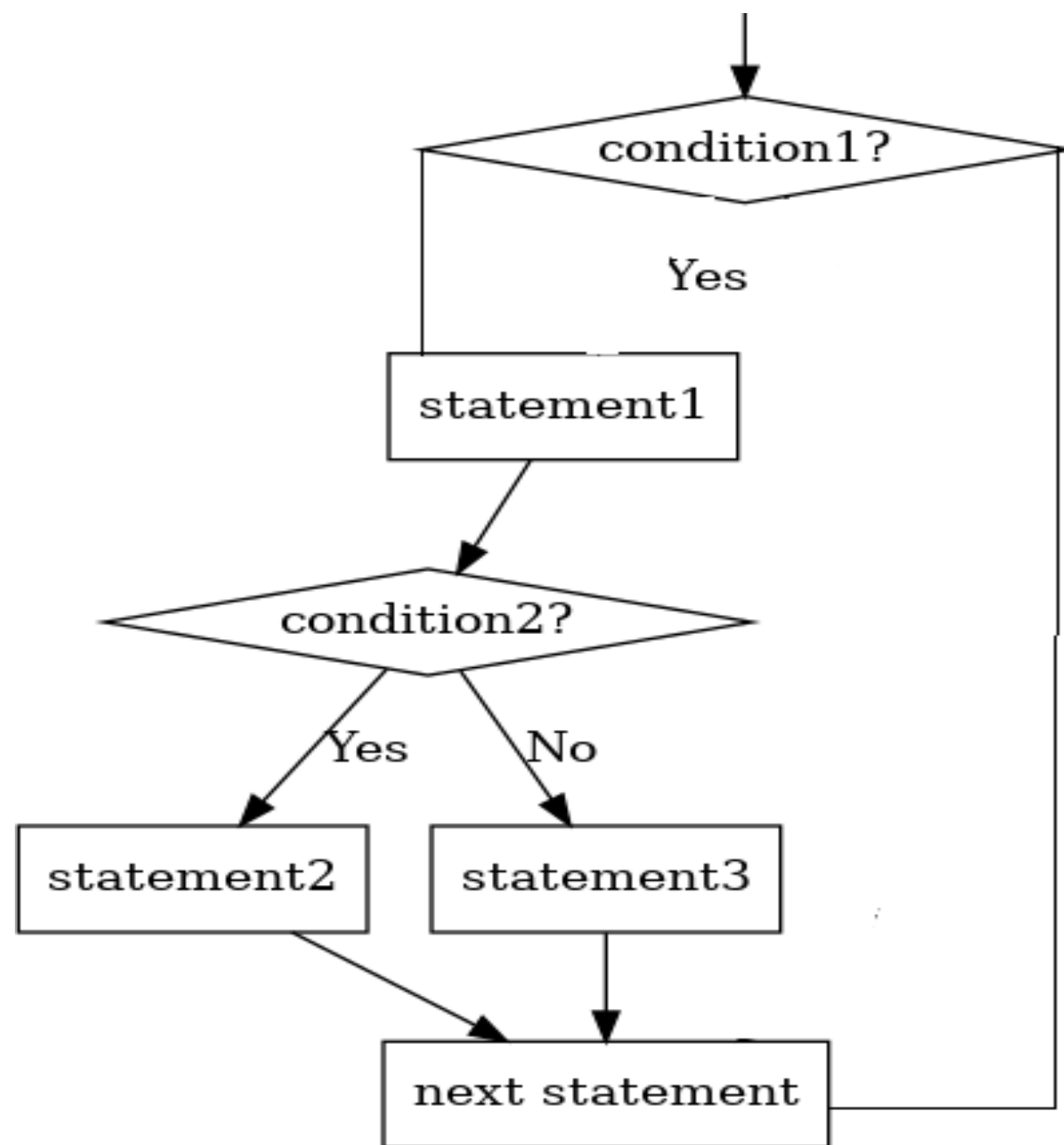
print("thank you")

Nested if:

Python supports **nested if statements** which means we can use a conditional if and if...else statement inside an existing **if statement**.

Syntax:

```
if condition1:  
    statement1  
    if condition2:  
        statement2  
    else:  
        statement3  
next statement
```



Example:

```
num = 36
```

```
print ("num = ", num)
```

```
if num % 2 == 0:
```

```
    if num % 3 == 0:
```

```
        print ("Divisible by 3 and 2")
```

```
    else:
```

```
        print ("Divisible by 2 but not by 3")
```

```
print("program ends here..")
```

Repetition Control Instruction

- 2 types of repetition control instructions:
 - while
 - for

For loop:

for is used to iterate over elements of a sequence such as string, tuple or list.

Syntax:

```
for value in sequence:  
    loop body
```

2 types:

```
for var in list :  
    statement1  
    statement2
```

```
for var in list :  
    statement1  
    statement2  
else :  
    statement3  
    statement4
```

- During each iteration **var** is assigned the next value from the list.
- In place of a list, a string, a tuple, set or dictionary can also be used.
- **else** block is optional.
- **If present**, it is executed if loop is not terminated abruptly using break.

```
x = 'nmamit'
```

```
for i in x:
```

```
    print(i)
```

output: ??

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```

Output:

```
apple  
banana  
cherry
```

Usage of for loop

- A for loop can be used in the following two situations:
 - Repeat a set of statements a finite number of times.
 - Iterate through a string, list, tuple, set or dictionary.
- To repeat a set of statements a finite number of times a built-in function **range()** is used.

- **range()** function generates a sequence of integers:
 - range(10)- generates a number from 0 to 9
 - range(10,20)- generates a number from 10 to 19
 - range(10,20,2)- generates a number from 10 to 19 in step of 2
 - range(20,10,-3)- generates a number from 20 to 9 in step of -3
- **range() cannot generate sequence of floats.**

➤ **range(start, stop, step)**

- Produces a sequence of integers from start(inclusive) to stop(exclusive) by step.

```
# Generate even numbers from 0 up to (but not including) 10
for number in range(0, 10, 2):
    print(number)
```

- **Example 1:**

```
for x in range(2, 30, 3):  
    print(x)
```

Output: 2 5 8 11 14 17 20 23 26 29

Example 2:

```
for i in range(5, -1, -1):  
    print(i)
```

Output: 5 4 3 2 1 0

- The list of numbers generated using **range()** can be iterated through using a for loop.

```
for i in range(1, 10, 2):  
    print(i, i * i, i * i * i)
```

- Output:

```
1  1  1  
3  9  27  
5  25 125  
7  49 343  
9  81 729
```

- for can be used to iterate through a string, list, tuple, set or dictionary as shown below:

```
for char in 'Leopard' :  
    print(char)  
for animal in ['Cat', 'Dog', 'Tiger', 'Lion', 'Leopard'] :  
    print(animal)  
for flower in ('Rose', 'Lily', 'Jasmine') :  
    print(flower)  
for num in {10, 20, 30, -10, -25} :  
    print(num)  
for key in {'A101' : 'Rajesh', 'A111' : 'Sunil', 'A112' : 'Rakesh'} :  
    print(key)
```

break statement



The break Statement

The break statement in Python is used to exit or “break” out of a loop (either a for or while loop) prematurely before the loop has iterated through all its items or reached its condition.

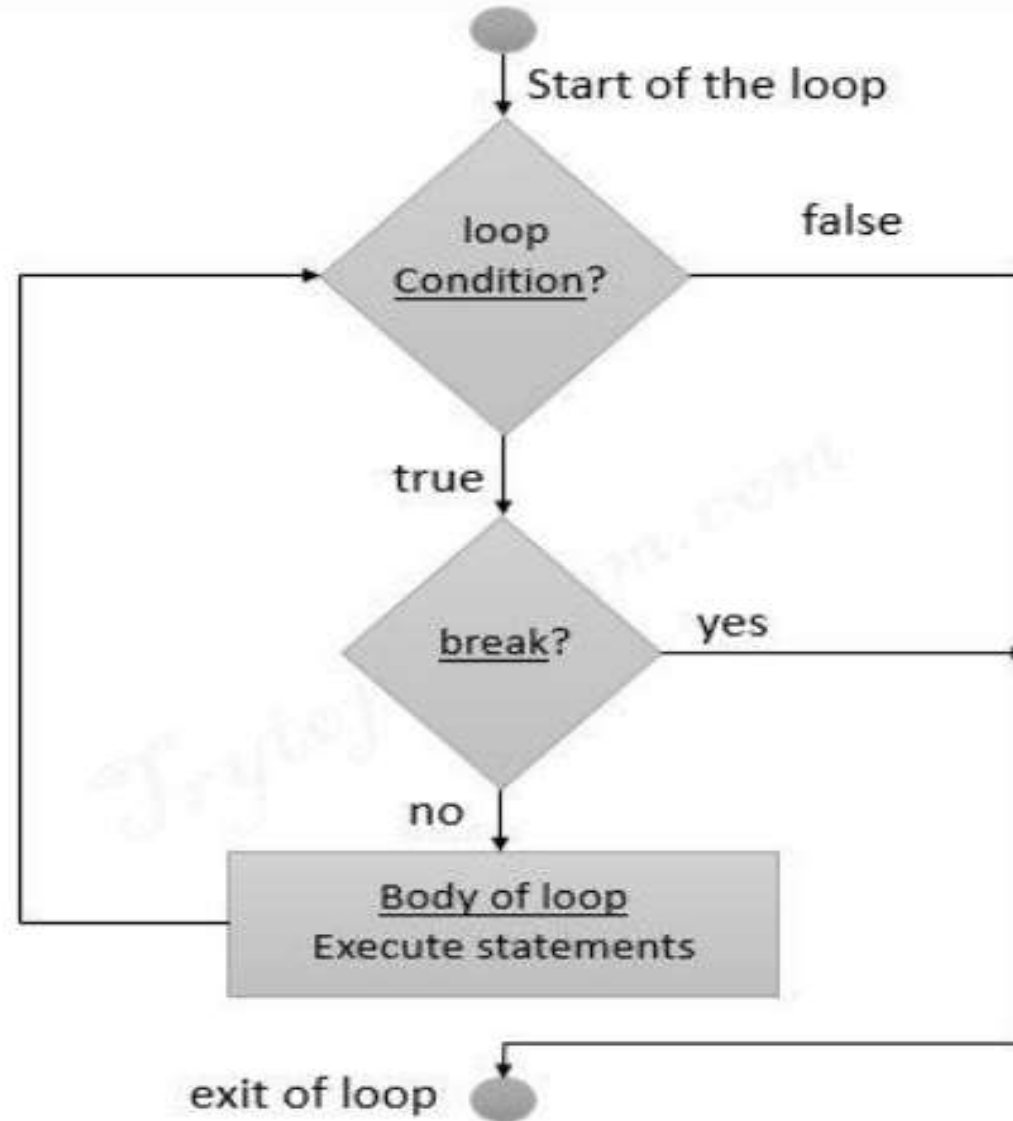
Example:

```
x = 'nmamit'
for val in x:
    if val=='i':
        break
    print(i)
```

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

Output:

Working of break statement



The continue Statement

With the continue statement we can stop the current iteration of the loop, and continue with the next.

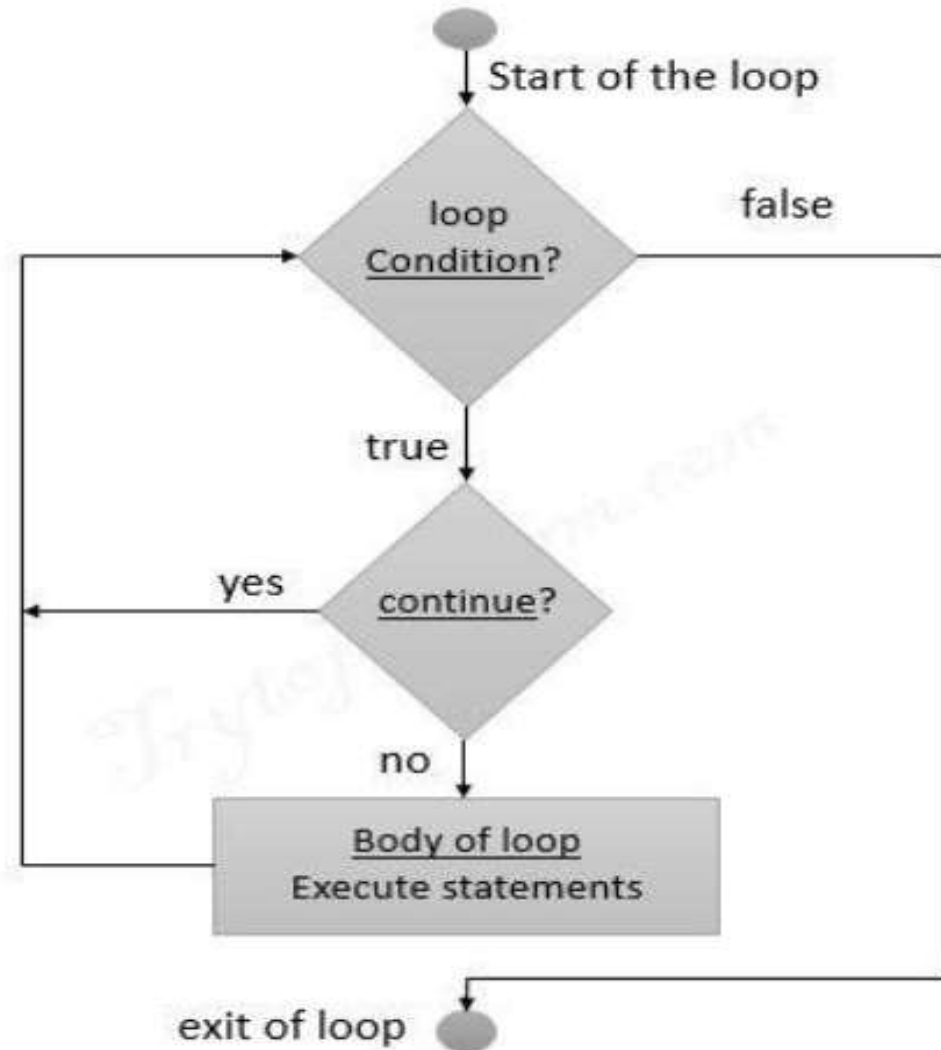
Python **continue statement** is used to skip the execution of the program block and returns the control to the beginning of the current loop to start the next iteration.

```
x = 'nmamit'
for i in x:
    if i=='i':
        continue
    print(i)
```

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

Output:

Working of continue statement:



Using else Statement with for Loop

- A loop expression and an else expression can be connected in Python.
- After the circuit has finished iterating over the list, the else clause is combined with a for Loop.

Example:

```
for x in range(6):
```

```
    if x == 3:
```

```
        break
```

```
        print(x)
```

```
else:
```

```
    print("finished!")
```

```
start = int(input("Enter starting range"))
end = int(input("Enter ending range"))
c=0
print("Prime numbers between", start, "and", end, "are:")

for num in range(start, end+1):
    # all prime numbers are greater than 1
    if num > 1:
        for i in range(2, num):
            if (num % i) == 0:
                break
        else:
            c=c+1
            print(num)
if (c==0):
    print("There are no prime numbers in the range")
```

while loop

A while loop in Python is a control flow statement used to repeat a block of code as long as a specified condition is True. It's especially useful when the number of iterations isn't known beforehand but depends on some dynamic condition.

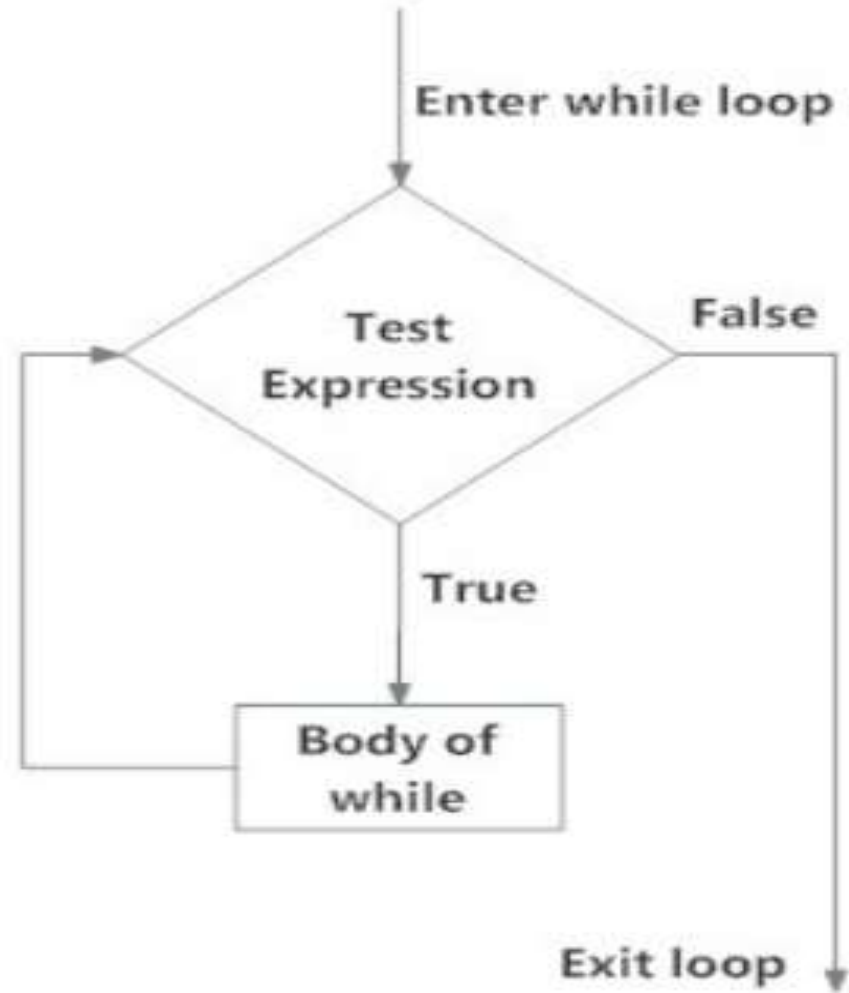
Syntax:

while expression:

statement(s)

- **condition:** This is a boolean expression. If it evaluates to True, the code inside the loop will execute.
- **statement(s):** These are the statements that will be executed during each iteration of the loop.

Working of while loop



Snippet 1: Print Numbers from 1 to 5

```
num = 1
while num <= 5:
    print(num)
    num += 1
```

Output:

1
2
3
4
5

Snippet 2: Sum of Numbers from 1 to 10

```
num = 1
total = 0
while num <= 10: # Condition: num should be 10 or less
    total += num # Add num to total
    num += 1 # Increment num by 1
print("Sum is:", total)
```

Output:

```
Sum is: 55
```

Snippet 3: Even Numbers Between 1 and 10

```
num = 1
while num <= 10: # Loop through numbers from 1 to 10
    if num % 2 == 0: # Check if num is even
        print(num) # Print even numbers
    num += 1        # Increment num by 1
```

Output:

2
4
6
8
10

Snippet 4: Print "Hello, World!" 5 Times

```
count = 1
while count <= 5: # Loop 5 times
    print("Hello, World!")
    count += 1
```

Output:

```
Hello, World!
Hello, World!
Hello, World!
Hello, World!
Hello, World!
```

Snippet 5: Countdown Timer

```
countdown = 5
while countdown > 0: # Condition: countdown should be greater than 0
    print(countdown)
    countdown -= 1 # Decrement countdown by 1
print("Liftoff!")
```

Output:

5

4

3

2

1

Liftoff!

Snippet 6: Infinite Loop

```
while True:  
    print("This will run forever!")
```

Output:

The string “This will run forever!” will be printed infinite times.

Syntax:

- Usage of break with for loop
- Usage of break with while loop

```
for val in sequence:  
    # code  
    if condition:  
        break  
    # code
```



```
while condition:  
    # code  
    if condition:  
        break  
    # code
```

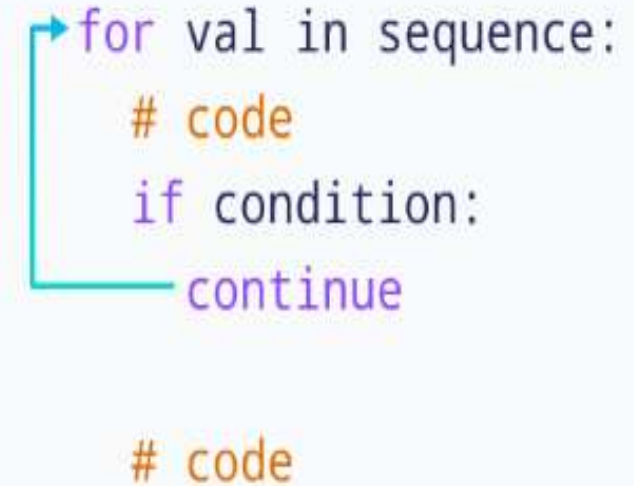


break statement

- The break statement in Python is used to exit or “break” out of a loop (either a for or while loop) prematurely before the loop has iterated through all its items or reached its condition. When the break statement is executed, the program immediately exits the loop, and the control moves to the next line of code after the loop.

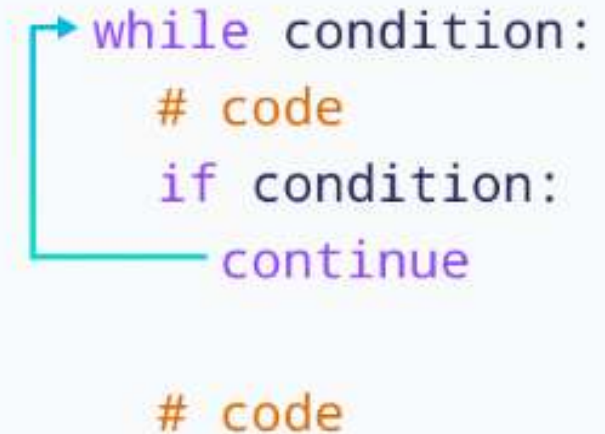
Syntax:

- Usage of continue with for loop



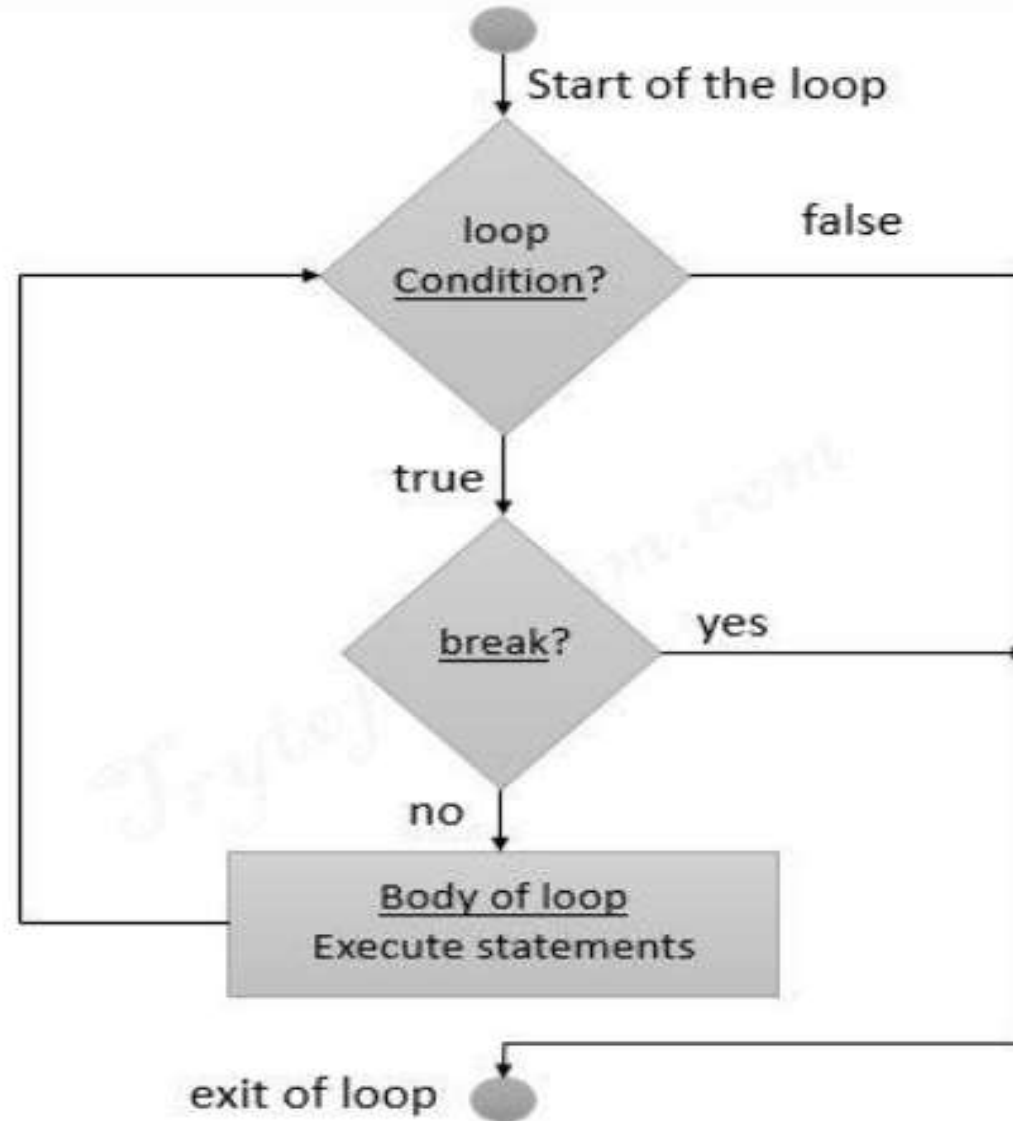
```
for val in sequence:  
    # code  
    if condition:  
        continue  
  
    # code
```

- Usage of continue with while loop



```
while condition:  
    # code  
    if condition:  
        continue  
  
    # code
```

Working of break statement



Snippet 1: Breaking a for loop

```
for i in range(5):  
    if i == 3:  
        break  
    print(i)
```

Output

0
1
2

=== Code Execution Successful ===

Snippet 2: Breaking a for loop

```
for number in range(1, 6):  
    if number == 3:  
        print("Breaking the loop at number:", number)  
        break  
    print("Current number is:", number)
```

Output

```
Current number is: 1  
Current number is: 2  
Breaking the loop at number: 3
```

```
=== Code Execution Successful ===
```

Snippet 3: Nested for loop with break

```
for i in range(3): # Outer loop
    for j in range(3): # Inner loop
        if j == 1:
            print(f"Breaking inner loop at i={i}, j={j}")
            break # Breaks only the inner loop
        print(f"i: {i}, j: {j}")
```

Output

```
i: 0, j: 0
Breaking inner loop at i=0, j=1
i: 1, j: 0
Breaking inner loop at i=1, j=1
i: 2, j: 0
Breaking inner loop at i=2, j=1
```

```
=== Code Execution Successful ===
```

Snippet 4: Breaking a while loop

```
count = 0
while count < 5:
    if count == 3:
        print("Breaking the loop at count:", count)
        break
    print("Count is:", count)
    count += 1
```

Output

```
Count is: 0
Count is: 1
Count is: 2
Breaking the loop at count: 3
```

```
=== Code Execution Successful ===
```

Snippet 5: Using break in an infinite loop

```
while True:
    user_input = input("Type 'stop' to exit: ")
    if user_input.lower() == 'stop':
        print("Exiting the loop.")
        break
    print("You entered:", user_input)
```

Output

```
Type 'stop' to exit: Hi
You entered: Hi
Type 'stop' to exit: Hello
You entered: Hello
Type 'stop' to exit: No
You entered: No
Type 'stop' to exit: Stop
Exiting the loop.
```

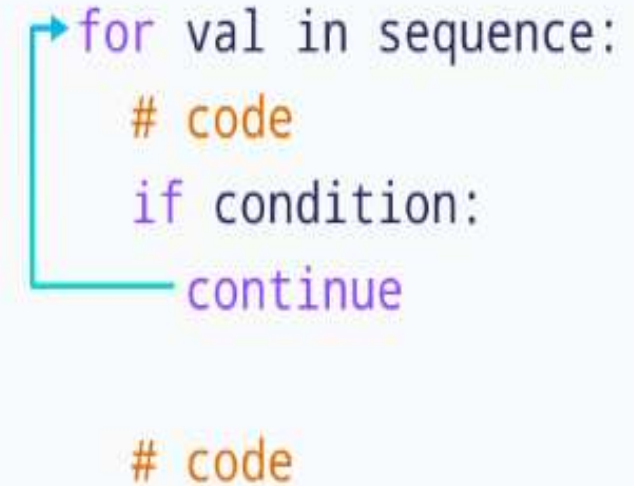
=== Code Execution Successful ===

continue statement

- Python **continue statement** is used to skip the execution of the program block and returns the control to the beginning of the current loop to start the next iteration. When encountered, the loop starts next iteration without executing the remaining statements in the current iteration.
- The **continue statement** is just the opposite to that of break. It skips the remaining statements in the current loop and starts the next iteration.

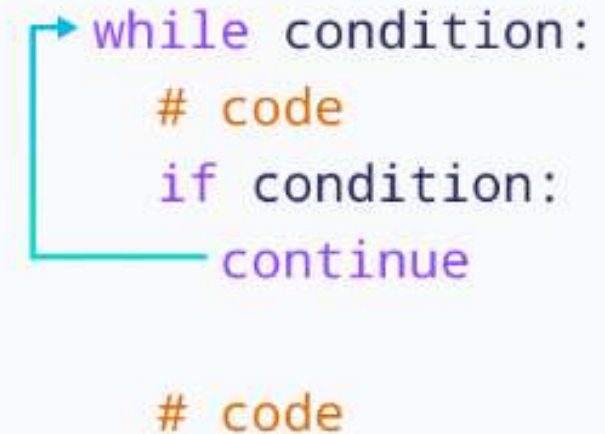
Syntax:

- Usage of continue with for loop



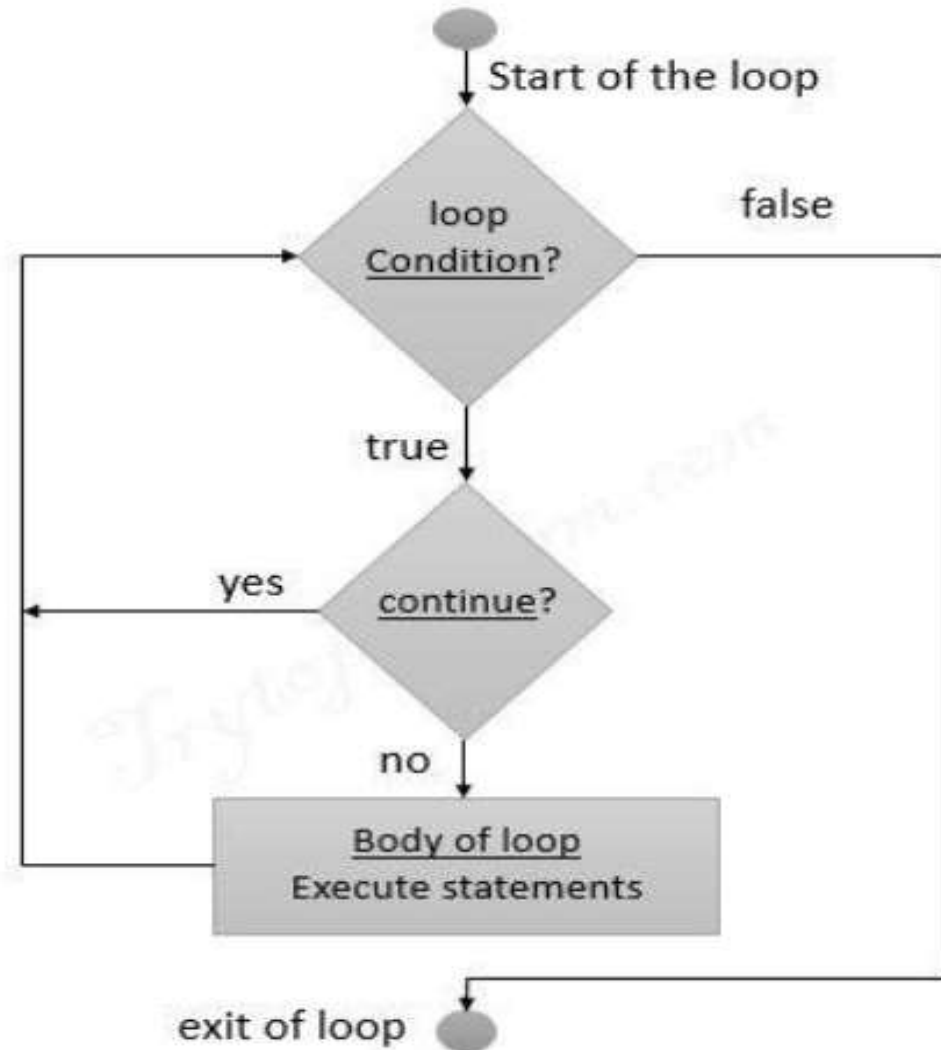
```
for val in sequence:  
    # code  
    if condition:  
        continue  
  
    # code
```

- Usage of continue with while loop



```
while condition:  
    # code  
    if condition:  
        continue  
  
    # code
```

Working of continue statement:



Snippet 1:

```
for i in range(5):  
    if i == 3:  
        continue  
    print(i)
```

Output

0
1
2
4

=== Code Execution Successful ===

Snippet 2:

```
for var in "PythonProgramming":  
    if var == "o":  
        continue  
    print(var, end=" ")
```

Output

```
P y t h n P r g r a m m i n g  
=== Code Execution Successful ===
```

Snippet 3: Skip Odd Numbers in a Loop

```
for num in range(1, 10): # Numbers from 1 to 9
    if num % 2 != 0:
        continue # Skip odd numbers
    print(num) # Only even numbers will be printed
```

Output

2
4
6
8

=== Code Execution Successful ===

Snippet 4:

```
# loop from 1 to 10
for i in range(1, 11):

    # If i is equals to 6,
    # continue to next iteration
    # without printing
    if i == 6:
        continue
    else:
        # otherwise print the value
        # of i
        print(i, end=" ")
```

Output

1 2 3 4 5 7 8 9 10

=== Code Execution Successful ===

Snippet 5:

```
i = 1
while i <= 10 :
    if i == 4 or i==7 :
        i += 1
        continue
    print(i)
    i += 1
```

Output

```
1
2
3
5
6
8
9
10
```

Snippet 6:

```
for x in range(1, 11) :  
    if x%3 == 0 :  
        continue  
    print(x)
```

Output

```
1  
2  
4  
5  
7  
8  
10
```

Snippet 7:

Output

```
i = 1, j = 1  
i = 1, j = 3  
i = 2, j = 1  
i = 2, j = 3  
i = 3, j = 1  
i = 3, j = 3
```

```
=== Code Execution Successful ===
```

```
for i in range(1, 4): # Outer loop  
    for j in range(1, 4): # Inner loop  
        if j == 2:  
            continue # Skip the rest of this iteration of the inner loop  
        print(f"i = {i}, j = {j}")
```

pass statement

- The Python pass statement is a null statement. But the difference between pass and comment is that comment is ignored by the interpreter whereas pass is not ignored.

Syntax:

pass

What is pass statement in Python?

When the user does not know what code to write, So user simply places a pass at that line. Sometimes, the pass is used when the user doesn't want any code to execute. So users can simply place a pass where empty code is not allowed, like in loops, function definitions, class definitions, or in if statements. So using a pass statement user avoids this error.

Using pass With Conditional Statement

```
n = 10
n=n+1
# use pass inside if statement
if n > 9:
    pass
else:
    print("Hello")
```

Output

Using pass With Conditional Statement

```
n = 10
n=n-1
# use pass inside if statement
if n > 9:
    pass
else:
    print("Hello")
```

Output

Hello

=== Code Execution Successful ===

Using pass With Looping Statement

Output

9

=== Code Execution Successful ===

```
n = 10
for i in range(n):

    # pass can be used as placeholder
    # when code is to added later
    pass
print(i)
```

Using pass With Looping Statement (while)

Output

```
Count is: 0  
Count is: 1  
Count is: 2  
Count is: 4
```

```
=== Code Execution Successful ===
```

```
count = 0  
  
while count < 5:  
    if count == 3:  
        # Do nothing when count is 3  
        pass  
    else:  
        print(f"Count is: {count}")  
    count += 1
```

Thank You